

中北大学软件学院

软件实践说明书

课程知识点答疑台

系统设计报告

(基于 RAG 的课程答疑与知识回溯系统)

学 院：软件学院

专 业：_____

班 级：_____

姓 名：_____

学 号：_____

指导教师：_____

日 期：_____

目 录

1 引言	1
1.1 设计目标与范围	1
1.2 设计原则	1
1.3 开发环境与工具	1
1.4 论文内容安排	1
2 系统总体架构设计	2
2.1 项目概述与业务目标	2
2.2 技术选型	2
2.3 架构风格与分层设计	2
2.4 系统总体架构图	3
2.5 部署与运行环境	3
3 系统模块详细设计	5
3.1 模块总体结构	5
3.2 资料载入与清洗模块 (loader + cleaner)	6
3.3 知识切分与索引模块 (splitter + embeddings + vectorstore)	6
3.4 检索模块 (retriever)	7
3.5 精排模块 (reranker)	7
3.6 问答生成模块 (generator, 含提示词设计)	7
3.7 接口与文档服务模块 (docs_service)	8
3.8 评测模块 (eval)	8
4 系统功能与交互设计	9
4.1 主要功能实现方案	9
4.2 接口设计	9
4.3 问答交互时序	9
4.4 关键界面设计	10
5 数据设计	11
5.1 数据组成	11

5.2 语料结构	11
5.3 向量库设计	11
5.4 语料指纹机制	11
5.5 数据存储关系	12
5.6 配置与安全	12
6 评测与验证	13
6.1 评测方案	13
6.2 测试问题清单	13
6.3 评测结果	14
6.4 结果分析与结论	14
7 结论	16
参 考 文 献	17
致 谢	18

1 引言

1.1 设计目标与范围

本报告依据《课程知识点答疑台需求分析报告》对系统进行总体与详细设计。系统以检索增强生成（RAG）为核心范式^[1]，将课程教学资料整理为可检索的知识库，学生以自然语言提问，系统基于检索片段生成分点讲解并展示依据来源。设计范围覆盖总体架构、模块设计、功能与交互设计、数据设计以及评测与验证。

1.2 设计原则

- （1）回答可溯源：生成模块仅依据检索命中的知识片段组织回答，每条结论对应可见来源。
- （2）检索是地板、优化分层：检索模块只负责“捞回候选”，不做过滤；清洗、拒答阈值等优化手段分层实现，不混入检索核心。
- （3）库是语料的函数：语料路径、切分参数、嵌入模型、清洗规则构成语料指纹，指纹不一致时向量库自动重建，避免脏数据。
- （4）先答后免责：生成策略要求模型先回答资料可支撑的部分，资料完全无法回答时才说明不知道，禁止以“资料中没有”开头。

1.3 开发环境与工具

开发语言 Python 3.11；AI 编排框架 LangChain^[2]；后端框架 FastAPI^[3]；向量数据库 Chroma^[4]；嵌入模型 jina-embeddings-v3^[5]（本地运行）；精排模型 bge-reranker-base^[6]（本地运行）；LLM 采用 OpenAI 兼容接口。

1.4 论文内容安排

第 2 章给出系统总体架构设计，第 3 章进行系统模块详细设计，第 4 章进行功能与交互设计，第 5 章进行数据设计，第 6 章给出评测与验证，第 7 章给出结论。

2 系统总体架构设计

2.1 项目概述与业务目标

本项目将一门课程的教学资料（课件、教材章节、习题与解答等）整理为可检索的知识库，学生用自然语言提问即可获得分点讲解，并同步展示答案所依据的资料名称与原文片段；系统对知识库未覆盖的问题明确说明，不得生成没有依据的内容。系统建成后形成从资料导入、问题检索、讲解生成到依据回查的完整应用流程。

2.2 技术选型

表 2.1 技术选型表

技术领域	选型	用途与说明
开发语言	Python 3.11	项目开发与业务逻辑实现
后端框架	FastAPI[3]	提供文档服务 REST 接口与问答接口
前端框架	Vue3 + Vite	单页应用：问答主页、阅读页、历史与笔记（开发中）
AI 编排	LangChain[2]	构建文档载入、切分、检索与问答链
向量数据库	Chroma[4]	存储知识片段向量并支持相似度检索
嵌入模型	jina-embeddings-v3[5]	文本向量化，query/passage 分流，本地运行
精排模型	bge-reranker-base[6]	交叉编码器对候选片段重排
LLM	OpenAI 兼容接口	基于检索片段生成分点讲解
语料	QACP 中文 Python 问答集[7]	534 篇课程问答语料

2.3 架构风格与分层设计

系统采用分层架构（Layered Architecture）风格，自上而下划分为表示层、接口层、业务逻辑层与数据层，各层职责单一、边界清晰，上层依赖下层、下层不感知上层，便于独立开发、测试与维护。核心业务采用 RAG 架构：检索模块从知识库中为用户问题召回最相关的知识片段，生成模块仅依据这些片段组织回答，从而实现“回答可溯源”的目标。

2.4 系统总体架构图

系统总体架构如图 2.1 所示。用户通过浏览器访问前端页面发起提问；请求经 FastAPI 接口层进入业务逻辑层；业务逻辑层由 rag_baseline 流水线完成“载入→清洗→切块→入盘→检索/改写→精排→生成”七个环节；数据层由语料目录、Chroma 向量库、语料指纹与配置文件构成；嵌入与精排模型本地运行，LLM 通过 OpenAI 兼容接口获得。

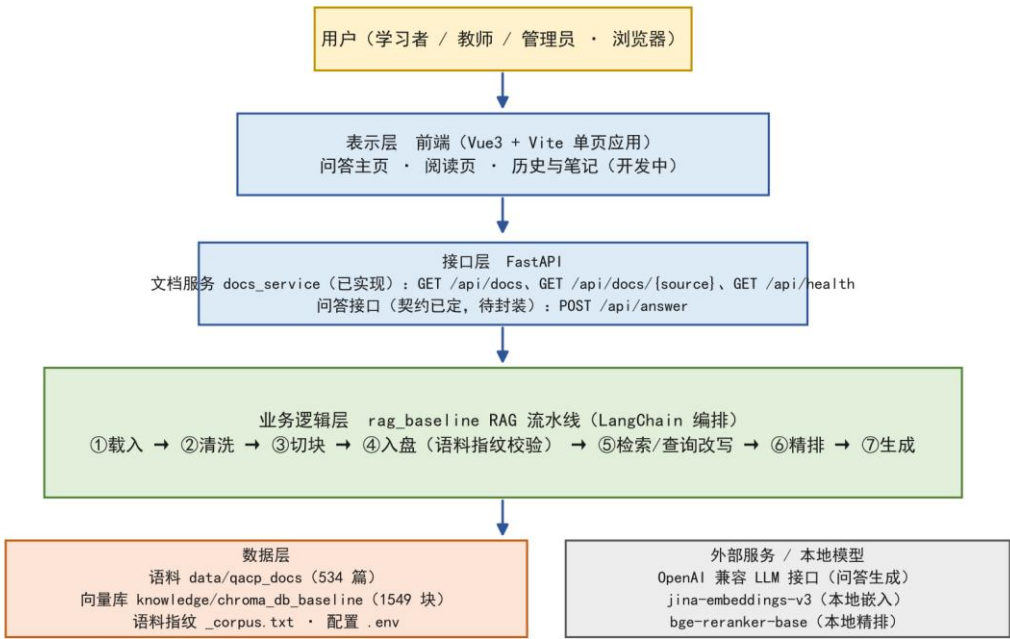


图 2.1 系统总体架构图

2.5 部署与运行环境

表 2.2 部署与运行环境

类别	要求/说明
操作系统	Windows / Linux / macOS 均可，建议 Windows 10 及以上
Python 版本	Python 3.11（项目指定版本）
内存	建议 4 GB 及以上
关键依赖	langchain、langchain-community、langchain-openai、chromadb、fastapi、uvicorn、fastembed、sentence-transformers、python-docx、pypdf、pydantic 等（详见 requirements.txt）
环境配置	通过 .env 文件提供 LLM_API_KEY、LLM_MODEL、LLM_BASE_URL

软件学院软件实践说明书

及向量库路径等参数

启动方式

文档服务： `python -m docs_service`（默认 127.0.0.1:8000）； 问答流水线：
`python run_baseline.py` “问题”

3 系统模块详细设计

3.1 模块总体结构

系统划分为接口服务（docs_service）、RAG 流水线（rag_baseline）与评测（eval）三大部分。rag_baseline 按流水线环节拆分为 config、embeddings、cleaner、loader、splitter、vectorstore、retriever、reranker、generator、main 共十个模块，各模块通过明确的接口协同工作，形成“资料导入→问题检索→精排→生成讲解→展示来源”的完整链路。系统模块结构如图 3.1 所示，各模块核心职责如表 3.1 所示。

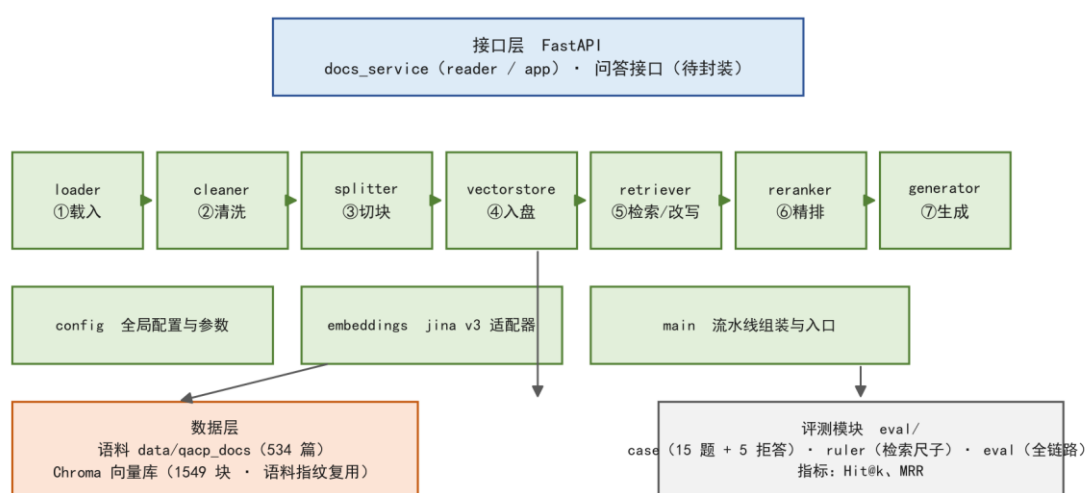


图 3.1 系统模块结构图

表 3.1 模块职责表

模块	核心职责
config	全局配置：语料/库路径、切分/检索/精排参数、环境变量
embeddings	jina v3 嵌入适配器，query/passage 任务分流
cleaner	语料清洗规则与清洗函数，剔除噪音标记
loader	载入：读文件→清洗→生成 Document
splitter	过滤空白块 + 递归切块（500 字符/重叠 50）
vectorstore	向量库构建与复用、语料指纹校验
retriever	向量检索 Top-k 候选 + 查询改写多路检索

reranker	交叉编码器精排，取 Top-5
generator	依据片段与提示词调用 LLM 生成回答
main	流水线组装与命令行入口
docs_service	FastAPI 文档服务：reader（校验/读取）、app（路由）
eval	评测：case（题集）、ruler（检索尺子）、eval（全链路）

3.2 资料载入与清洗模块（loader + cleaner）

（1）模块职责。loader 按语料目录模式读取全部 Markdown 文件，跳过非文档文件（以下划线开头等），并过滤清洗后不足 80 字的空页；cleaner 按预定义清洗规则（frontmatter、锚点、源码引用、提示框标签与边界、HTML 标签、markdown 链接）逐条清洗，行内代码与代码块作为“信号”原样保留。

（2）关键接口。clean_md(text): 按 CLEAN_RULES 顺序清洗并压缩连续空行；is_corpus_noise_file(name): 判定非文档文件；load_docs() -> list[Document]: 读取全部语料，返回带 source 元数据的文档列表。

（3）处理流程。扫描语料目录 → 过滤非文档文件 → 读取原文 → 清洗 → 长度过滤 → 输出 Document 列表。

3.3 知识切分与索引模块（splitter + embeddings + vectorstore）

（1）模块职责。将清洗后的文档按递归分隔符（段落→句子→字符）切分为知识片段，默认 chunk_size=500、chunk_overlap=50；嵌入模型将片段向量化；向量库负责写入与检索。

（2）关键接口。RecursiveCharacterTextSplitter: LangChain 递归文本切分器；MyEmbeddings: jina v3 适配器（embed_documents / embed_query）；corpus_signature(): 生成语料指纹；build(chunks): 重建整库并写入指纹；get_db(chunks): 指纹一致则复用现成库，否则重建。

（3）处理流程。文档列表 → 切块 → 过滤空白块 → 指纹校验 → 建库或复用 → 输出 Chroma 向量库。

3.4 检索模块 (retriever)

(1) 模块职责。接收用户问题，调用向量库执行相似度检索，返回 Top-k 个最相关片段及其距离得分；对含多个意图的复合问题先进行查询改写（只拆分、不换说法），分路检索后按块去重合并。

(2) 关键接口。`retrieve(db, question, k)`: 单路向量检索；`rewrite(llm, question)`: 查询改写，返回子问题列表（首项恒为原问题，保证只多捞不少捞）；`retrieve_multi(db, questions, k)`: 多路检索合并，同一片段保留距离最小的一路。

(3) 处理流程。问题向量化 → 检索 Top-k → (可选) 改写后多路检索合并 → 输出候选片段。检索不做阈值过滤，拒答守卫由下游按相关度判定。

3.5 精排模块 (reranker)

(1) 模块职责。用交叉编码器 (`bge-reranker-base`) 对检索候选重新打分排序，区分“语义沾边”与“就是它”，将最相关片段提升至首位。

(2) 关键接口。`get_reranker()`: 首次使用时惰性加载模型；`rerank(question, hits, top_n=5)`: 将 (问题, 片段) 拼接送入模型全量重打分，取前 5 条。

(3) 处理流程。候选片段 → (问题, 片段) 拼接 → 交叉编码器打分 → 降序取前 5 条 → 输出精排结果。

3.6 问答生成模块 (generator, 含提示词设计)

(1) 模块职责。基于精排后的命中片段与业务提示词调用 LLM 生成回答，遵循“先答后免责”策略：先回答资料可支撑的部分，资料完全无法回答时才说明不知道。

(2) 业务提示词设计。提示词在 `generator.py` 中构造，设计要点：明确回答要求（先直接回答、完整给出资料中的步骤/代码/要点）；边界处理（资料只覆盖一部分时先答已有部分，未覆盖内容最后补一句说明，禁止以“资料中没有”开头）；内容约束（全部出自资料，不自行补充资料之外的内容）。

(3) 关键接口。`generate(llm, question, hits) -> str`: 拼接上下文与提示词，调用 LLM 生成回答文本。

3.7 接口与文档服务模块 (docs_service)

(1) 模块职责。基于 FastAPI 提供文档阅读服务：语料文件列表、单篇文档全文与健康检查；文档名校验采用严格白名单正则，防止路径穿越。

(2) 关键接口。GET /api/health: 健康检查，返回 “status: ok”；GET /api/docs: 语料文件列表，按文件名序号自然排序；GET /api/docs/{source}: 返回文档全文，source 须为 q 数字.md 格式，非法名返回 400，文件不存在返回 404。

(3) 处理流程。请求 → 白名单校验（先格式后存在性）→ 读取全文（UTF-8 优先、GB18030 兜底）→ 提取标题 → 返回 JSON。

3.8 评测模块 (eval)

(1) 模块职责。内置固定测试集（15 条库内问题 + 5 条库外拒答问题），逐题运行检索/生成流程，统计引用命中率 Hit@k、平均倒数排名 MRR 与拒答率，输出评测报告。

(2) 关键接口。ruler.first_rank(hits, expected): 判定期望文档在检索结果中的排名；eval 全链路脚本：按阶段输出载入、切块、检索、精排、生成的独立指标。

(3) 处理流程。固定题集 → 逐题检索（可含改写与精排）→ 判定排名 → 汇总 Hit@k 与 MRR → 写入评测报告。

4 系统功能与交互设计

4.1 主要功能实现方案

- (1) 知识库构建：资料上传后自动完成解析、清洗、切块与向量化入库，语料指纹保证向量库与语料一致。
- (2) 知识点答疑与来源展示：学生以自然语言提问，系统经“检索→精排→生成”返回分点讲解，回答下方展示命中的来源文件名、原文片段与相关度。
- (3) 无依据问题处理：检索无结果或相关度低于阈值时输出“未找到依据”，不生成无来源的补充内容。
- (4) 文档阅读：学生点击来源卡片跳转阅读页，查看完整资料原文，支持 Markdown 渲染。
- (5) 历史记录与多轮追问：前端本地保存提问历史；后端携带历史上下文回答追问（扩展功能）。
- (6) 结果记录与评测：评测模块运行测试集并输出 Hit@k、MRR 与拒答率统计。

4.2 接口设计

表 4.1 主要接口清单

接口	方法	说明	状态
/api/health	GET	健康检查，返回服务状态	已实现
/api/docs	GET	获取语料文件列表	已实现
/api/docs/{source}	GET	获取单篇文档全文	已实现
/api/answer	POST	提交问题，返回讲解与来源列表	待封装（契约已定）

4.3 问答交互时序

问答交互时序如图 4.1 所示：用户输入问题后，前端调用问答接口，后端依次完成向量检索、精排、LLM 生成，最后将分点讲解与来源列表返回前端渲染。

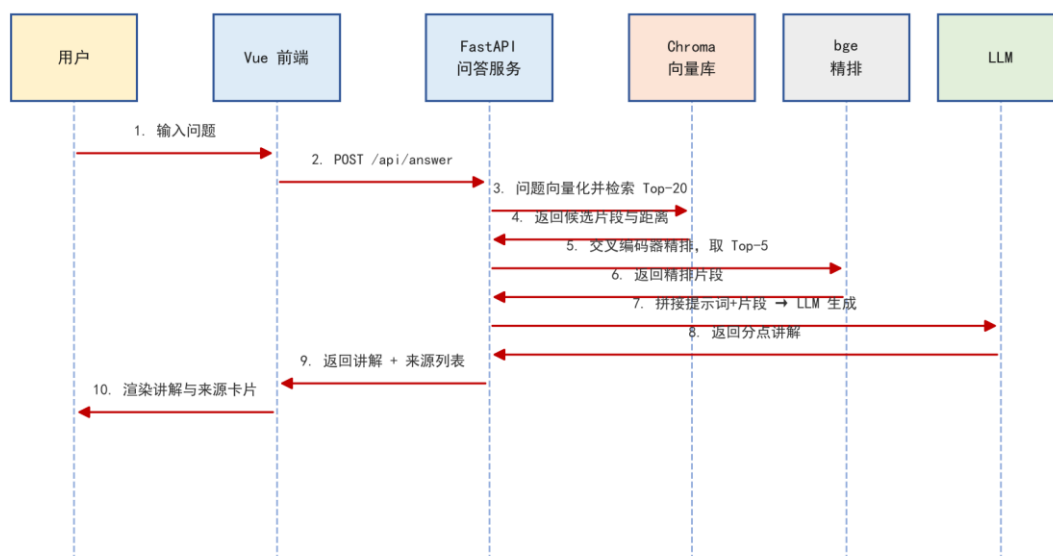


图 4.1 问答交互时序图

4.4 关键界面设计

- (1) 问答主页：问题输入框、发送按钮、答案展示区（Markdown 渲染）与来源卡片列表（文件名、相关度、片段预览），点击来源卡片跳转阅读页。
- (2) 阅读页：展示语料文件列表与单篇文档全文，支持代码高亮。
- (3) 历史与笔记：本地保存提问历史，支持回看、存笔记与导出。

5 数据设计

5.1 数据组成

系统数据由四部分组成：语料目录（data/qacp_docs）、向量库（knowledge/chroma_db_baseline）、语料指纹（_corpus.txt）与配置文件（.env），以及评测报告（eval/last_run.txt、eval/pipeline_last_run.txt）。

5.2 语料结构

表 5.1 语料结构说明

项目	说明
数据来源	QACP 中文 Python 问答集[7]，一篇问答一个文件
文件命名	q001.md ~ q534.md，共 534 篇，覆盖 10 个知识点
原始数据	data/_qacp_raw/（QACP_A.csv、QACP_Q.json），由 tools/build_qacp_corpus.py 转换生成
编码	原始 CSV 为 GB18030，转换后语料统一为 UTF-8

5.3 向量库设计

表 5.2 向量库（Chroma）结构

项目	说明
存储目录	knowledge/chroma_db_baseline
片段规模	1549 块（平均约 365 字符/块）
切分参数	chunk_size=500, chunk_overlap=50
嵌入模型	jina-embeddings-v3（query/passage 任务分流）
记录元数据	source: 来源文件名（相对路径）
检索方式	向量相似度检索（L2 距离），默认候选 Top-20

5.4 语料指纹机制

向量库是语料的函数。corpus_signature() 由“语料路径|文件模式|切分参数|嵌入模型|清洗规则”构成；库目录中的 _corpus.txt 记录建库时的指纹。每次启动先比对指纹：一致则复用现成库（秒级），不一致则自动重建（本机约 12~15 分钟，仅建一次）。该机制保证语料或参数变化后不会检索到过期内容。

5.5 数据存储关系

系统数据流转与存储关系如图 5.1 所示。

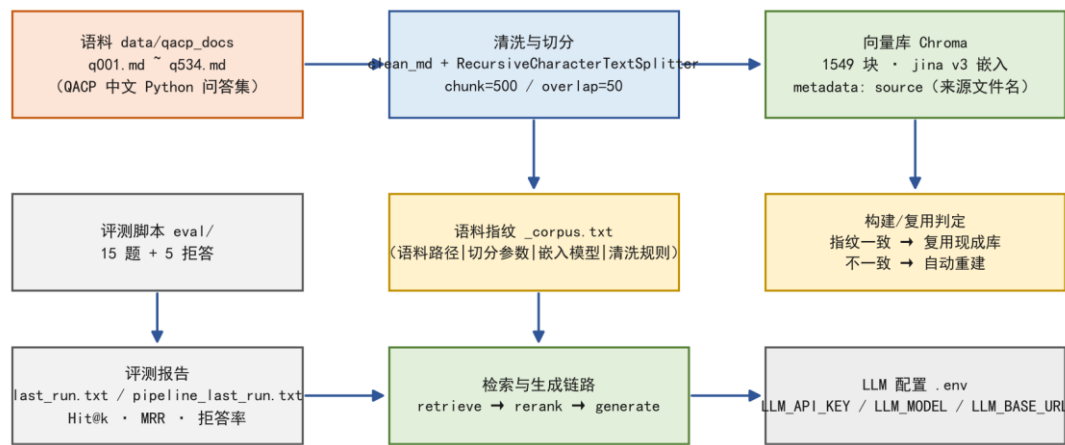


图 5.1 数据存储关系图

5.6 配置与安全

- (1) 模型密钥：LLM_API_KEY、LLM_MODEL、LLM_BASE_URL 通过 .env 文件配置管理，文件不提交代码仓库。
- (2) 文档读取白名单：docs_service 对文件名做正则全匹配 (^q\d+\.md\$)，并在路径解析后再校验一次是否仍在语料目录内，纵深防御路径穿越。

6 评测与验证

6.1 评测方案

评测采用固定测试集与固定指标：

(1) 测试集：15 条库内问题（A 组“词面直白”5 条、B 组“换个说法”5 条、C 组“口语化”5 条），每条标注期望命中的语料文件；另设 5 条库外拒答问题（含 2 条与 Python 高度接近的“陷阱题”）。

(2) 指标：引用命中率 Hit@k（期望文档进入前 k 名，k 取 1/3/5/10/20）、平均倒数排名 MRR、拒答率。

(3) 对比口径：以单路纯向量检索为基线，对比“检索+查询改写+精排”的全链路效果。

6.2 测试问题清单

表 6.1 库内测试问题清单（15 条）

序号	分组	测试问题	期望命中文件
1	A	什么是字符串？	q195.md
2	A	如何捕获异常？	q499.md
3	A	如何在列表中添加或删除元素？	q230.md、q231.md
4	A	默认参数是什么？	q306.md
5	A	如何定义一个类？	q404.md
6	B	怎么把多个字符串拼成一个？	q209.md
7	B	读文件报错时，怎么让程序不崩？	q469.md
8	B	想用别人写好的功能包，怎么引进来？	q337.md
9	B	写循环时怎么中途停住？	q137.md
10	B	类的构造方法怎么写？	q354.md
11	C	循环跑到一半想不跑了，直接出来，怎么写？	q187.md
12	C	我有个列表，里面有些东西重复了，想只留一份	q269.md
13	C	程序打开一个不存在的文件直接就崩了，能不能让它别崩？	q467.md
14	C	别人写好的类，我想在它基础上加点自己的东西	q413.md
15	C	两个变量想确认它们是不是同一个对象，用 == 行不行？	q109.md

表 6.2 库外拒答问题清单（5 条）

序号	测试问题	说明
1	红烧肉怎么做才好吃？	语义无关，应拒答
2	推荐几部好看的科幻电影	语义无关，应拒答
3	怎么用 Excel 做数据透视表？	语义无关，应拒答
4	怎么用 Python 写一个贪吃蛇游戏？	陷阱题（Python 话题），应拒答
5	怎么用 Python 自动给用户发邮件？	陷阱题（Python 话题），应拒答

6.3 评测结果

在项目实测环境中（向量库 1549 块、15 条测试问题）运行评测，结果如表 6.3 所示。

表 6.3 检索与精排评测结果

指标	纯向量检索（基线）	检索+改写+精排（全链路）
Hit@1	40.0%（6/15）	53.3%（8/15）
Hit@3	60.0%（9/15）	66.7%（10/15）
Hit@5	80.0%（12/15）	73.3%（11/15）
Hit@10	86.7%（13/15）	86.7%（13/15）
Hit@20	93.3%（14/15）	93.3%（14/15）
MRR	0.544	0.627

6.4 结果分析与结论

（1）精排显著提升首位命中率与整体排序质量：Hit@1 由 40.0% 提升至 53.3%，MRR 由 0.544 提升至 0.627（净收益 +0.084），说明交叉编码器能把“语义最相关”的片段顶到首位。

（2）Hit@5 由 80.0% 回落至 73.3%：精排按相关度重新排序后，部分期望片段虽仍在 Top-20 内但排名下移；首位命中率与 MRR 上升表明排序质量总体改善，该代价可接受。

（3）分组看，B 组（换个说法）基线 MRR 最低（0.292），是后续优化（查询改写、同义检索等）的重点方向。

（4）拒答策略通过相关度阈值守卫实现，5 条库外问题（含陷阱题）需配合阈值标定进一步验证。

(5) 端到端生成正确率需开启 CHECK_ANSWER 后另行评测，属后续工作。

7 结论

本项目完成了“课程知识点答疑台”的系统设计，实现并验证了以 RAG 为核心的问答链路：语料载入与清洗、知识切分与向量索引、向量检索与查询改写、交叉编码器精排、基于片段的 LLM 生成，以及文档阅读服务与检索质量评测。实测结果表明，精排将首位命中率由 40.0% 提升至 53.3%、MRR 由 0.544 提升至 0.627，“回答可溯源”与“无依据拒答”两大约束在设计中得到落实。

目前仍存在待完善之处：前端页面与问答接口封装尚未完成（接口契约已确定）；端到端回答质量评测（生成环节）尚未开启；多轮会话、用户权限等扩展功能留待后续迭代。下一阶段将完成前端与接口联调，并基于评测指标持续优化检索与生成链路。

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33 (NeurIPS 2020). 2020.
- [2] LangChain. LangChain Python Documentation[EB/OL]. <https://python.langchain.com/>.
- [3] FastAPI. FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com/>.
- [4] Chroma. Chroma Documentation[EB/OL]. <https://docs.trychroma.com/>.
- [5] Jina AI. jina-embeddings-v3: Multilingual Multimodal Embeddings[EB/OL]. <https://jina.ai/embeddings/>.
- [6] BAAI. bge-reranker-base[EB/OL]. <https://huggingface.co/BAAI/bge-reranker-base>.
- [7] NTAIX. Chinese-Python-QA-Dataset (QACP)[EB/OL]. <https://github.com/NTAIX/Chinese-Python-QA-Dataset>.

致 谢

本报告在撰写过程中得到了指导教师_____的悉心指导，老师在选题、系统设计思路与文档规范等方面给予了耐心帮助；同时感谢课程实践小组各位成员在需求梳理、后端实现与评测验证中的密切配合。在此一并致以诚挚的谢意。